

University
Politehnica
of Bucharest





Microsoft SQL Server (I)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro



Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

Microsoft SQL Server Overview

- Microsoft SQL Server (MSSQL) is a Relational Database Management System (RDBMS) capable of processing large amounts of data preparing for the big-data era
- MSSQL2022 stable release
- MSSQL2019 is **closed-source**
- MSSQL2019 is written in C/C++ like most of the Microsoft (core) products

Rank			DBMS	Database Model	Score		
Nov 2023	Oct 2023	Nov 2022			Nov 2023	Oct 2023	Nov 2022
1.	1.	1.	Oracle	Relational, Multi-model	1277.03	+15.61	+35.34
2.	2.	2.	MySQL	Relational, Multi-model	1115.24	-18.07	-90.30
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	911.42	+14.54	-1.09
4.	4.	4.	PostgreSQL	Relational, Multi-model	636.86	-1.96	+13.70
5.	5.	5.	MongoDB	Document, Multi-model	428.55	-2.87	-49.35
6.	6.	6.	Redis	Key-value, Multi-model	160.02	-2.95	-22.03
7.	7.	7.	Elasticsearch	Search engine, Multi-model	139.62	+2.48	-10.70
8.	8.	8.	IBM Db2	Relational, Multi-model	136.00	+1.13	-13.56
9.	9.	↑ 10.	SQLite	Relational	124.58	-0.56	-10.05
10.	10.	↓ 9.	Microsoft Access	Relational	124.49	+0.18	-10.53



Microsoft SQL Server Overview

- 1989 – Microsoft SQL Server started as a project to port Sybase SQL Server onto OS/2
- 1993 – SQL Server 4.2 is released for Windows NT
- 1995 – SQL Server 6.0 is released: end of collaboration with Sybase
- 1998 – SQL Server 7.0 is released: the source code is converted from C to C++
- 2005 – SQL Server 2005 is released: complete revision of the old Sybase code into Microsoft code
- 2017 – SQL Server 2017 adds Linux support for
  - Red Hat Enterprise Edition Linux
  - SUSE Linux Enterprise Edition
  - **Ubuntu**
  - **Docker Engine**

Microsoft SQL Server Overview

- Current versions still supported by Microsoft (Nov 2023):
  - SQL Server 2014
  - SQL Server 2016
  - SQL Server 2017
  - SQL Server 2019
  - SQL Server 2022 (current)

- From SQL Server 2016 support of x64 processors only

Microsoft SQL Server Overview

- SQL Server 2022 (like 2019) has 5 different editions
- The differences between them address
  - Scalability
  - Transparency
  - Performance
- Editions:
  - **Enterprise:** best for business (unlimited scaling)
  - **Standard:** ideal for small business (moderate scaling)
  - **Express:** portable version (lowest scaling)
  - **Web:** a low total-cost-of-ownership option for Web hosting
  - **Developer:** non-commercial (scale as much as the machine)

Microsoft SQL Server Overview

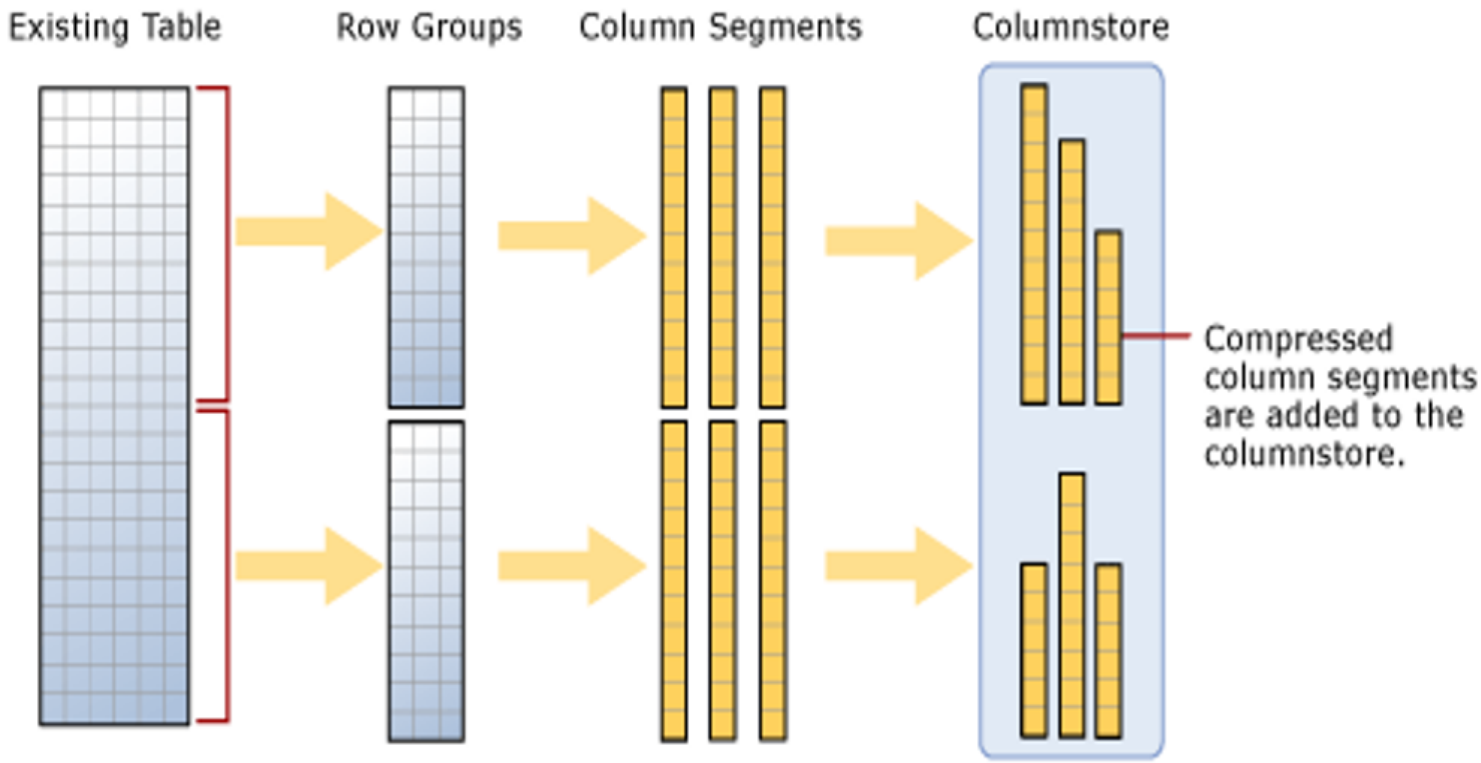
- Microsoft SQL Server is **cloud-ready**
  - Offered as Database-As-A-Service (**DaaS**) on multiple cloud providers
  - In Azure (Microsoft's Cloud Platform) it has a special flavour called Azure SQL Database
- SQL Server Big Data Cluster enabled with the release of Microsoft SQL Server 2019 (will be retired in Feb 2025)

Microsoft SQL Server Overview

- Microsoft SQL Server Features
  - Increase Performance
  - High-Availability
  - Security
  - Replication

Microsoft SQL Server Overview
Increase Peformance

- Columnstore indexes accelerate analytics performance by redefining data storage and queries



Microsoft SQL Server Overview
Increase Peformance

- **In-Memory OLTP** achieves greater speed through memory-optimized tables
- **Adaptive Query Processing** shapes optimization strategies based on application runtime conditions and characteristics
- **Automatic Database Tuning** corrects query execution plans causing performance issues
- **Query Store** gives insights on query plan choice and performance



Microsoft SQL Server Overview
High-Availability

- Always On Failover Cluster Instances and Availability Groups enable HADR (High-Availability Disaster Recovery) scenarios on both Linux and Windows
  - Always on Availability Groups feature is a HADR solution that provides an enterprise-level alternative to database mirroring.
  - Always on Failover Cluster Instances leverages Windows Server Failover Clustering (WSFC) functionality to provide local high availability through redundancy at the server-instance level-a *failover cluster instance* (FCI)
  - Up to eight secondary replicas are supported, including one primary replica and four synchronous-commit secondary replicas

- Read-Scale Availability Groups provide clusterless support for read-only workloads

- Database mirroring maintains two copies of a single database that must reside on different server instances



Microsoft SQL Server Overview
Security

- Database snapshot is a read-only, static view of a SQL Server database
- Dynamic Data Masking limits data exposure while simplifying security design and coding
- Row-Level Security controls access to rows in a database table based on user characteristics
- Always Encrypted provides separation between those who own and those who manage data
- Backup encryption gives extra security for backup files
- Backup compression used to compress backups which minimizes the I/O to increase backup speed significantly

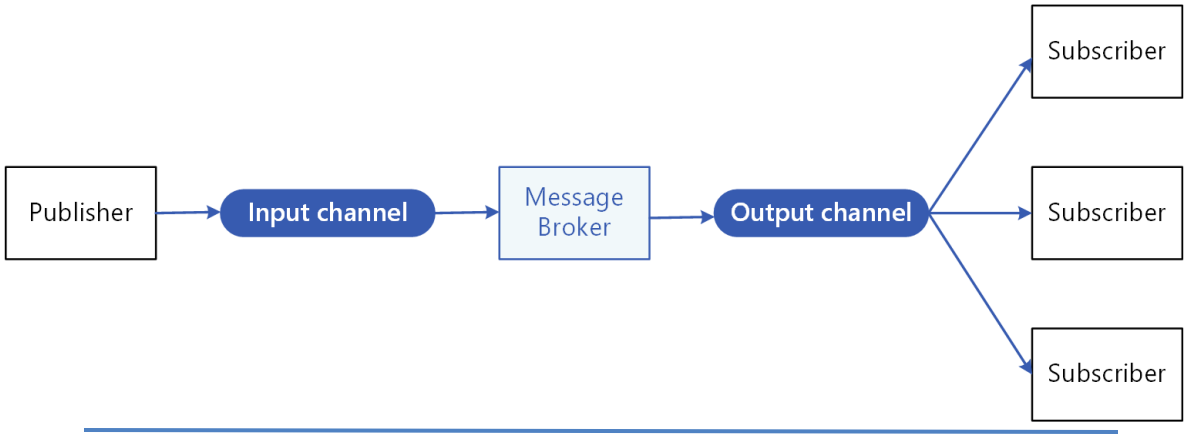
Microsoft SQL Server Overview
Replication

- Replication is a set of technologies for copying and distributing data and database objects from one database to another and then synchronizing to maintain consistency

- SQL Server replication types
  - Snapshot replication
  - Transactional replication
  - Merge replication

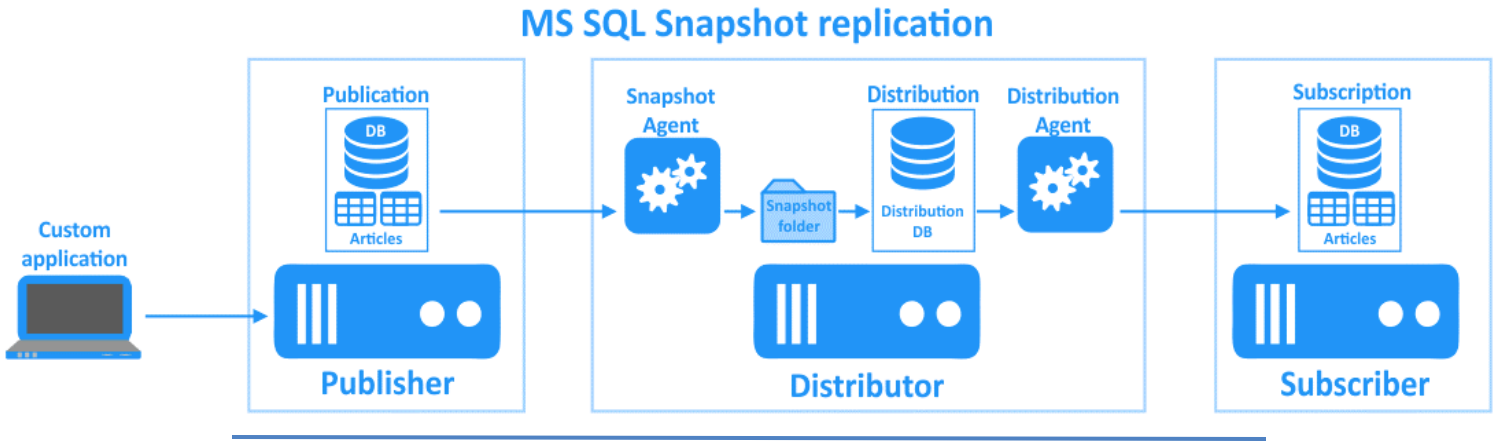
Microsoft SQL Server Overview
Publisher – Subscriber pattern

- Publisher-Subscriber is a design pattern that enables an application to announce events to many interested consumers asynchronously, where:
  - The publisher creates and sends messages/events
  - Subscribers register their interest in receiving some or all messages/events
  - The message broker (distributor) controls the interaction between the publishers and subscribers
    - The publisher post messages/events to the message broker
    - The subscribers register with the broker the messages/events they are interested in receiving
    - The broker stores and forwards messages/events from publishers to subscribers by filtering them by the subscribers' interests



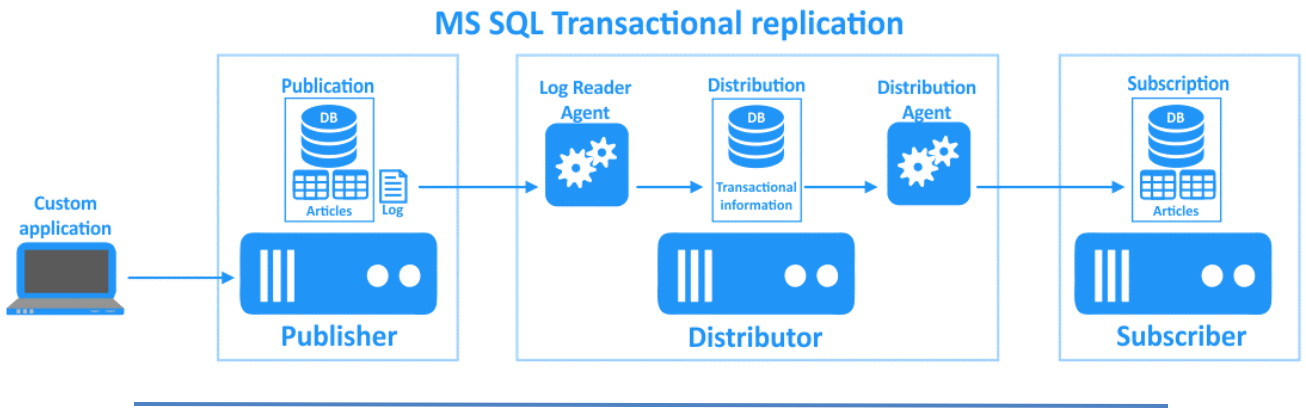
Microsoft SQL Server Overview
Replication

- Snapshot replication
  - Distributes data exactly as it appears at a specific moment in time
  - Does not monitor for updates to the data
  - When synchronization occurs, the entire snapshot is generated and sent to Subscribers



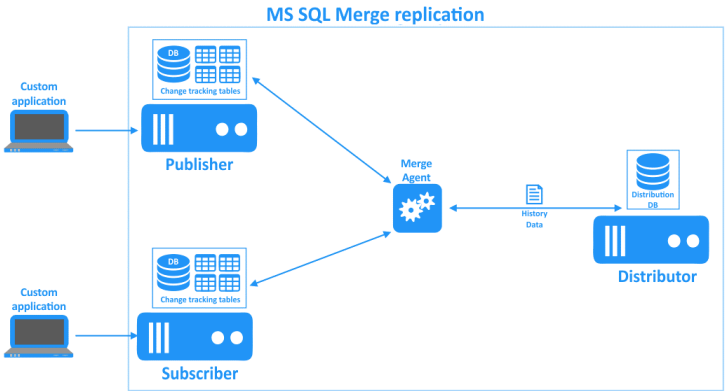
Microsoft SQL Server Overview
Replication

- Transactional replication
  - Starts with a snapshot of the publication database objects and data
  - After the initial snapshot subsequent data changes and schema modifications made at the Publisher are usually delivered to the Subscriber as they occur (in near real time)
  - The data changes are applied to the Subscriber in the same order and within the same transaction boundaries as they occurred at the Publisher



Microsoft SQL Server Overview
Replication

- Merge replication
  - Starts with a snapshot of the publication database objects and data
  - Subsequent data changes and schema modifications made at the Publisher and Subscribers are tracked with triggers
  - The Subscriber
    - Synchronizes with the Publisher when connected to the network
    - Exchanges all rows that have changed between the Publisher and Subscriber since the last time synchronization occurred

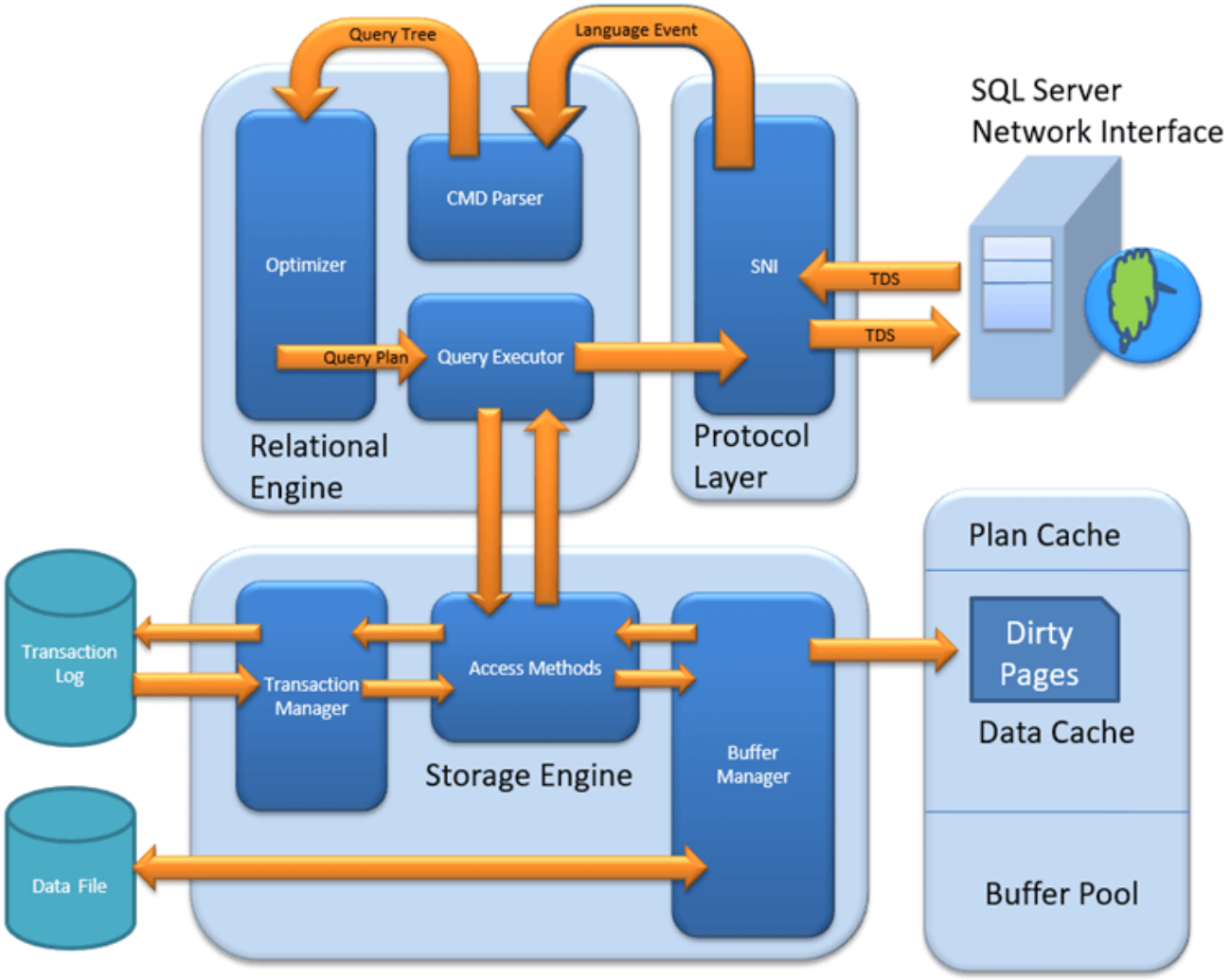




Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

Microsoft SQL Server Architecture



Microsoft SQL Server Architecture

- Microsoft SQL Server Main components:
  - Protocol Layer
  - Relational Engine
  - Storage Engine

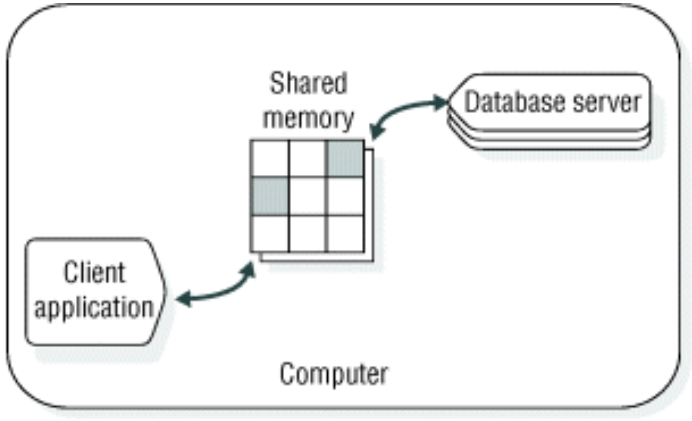
Microsoft SQL Server Architecture
Protocol Layer

- Microsoft SQL Server Network Interface (SNI)
  - Is a protocol layer that establishes the network connection between a client and the server
  - Consists of a set of APIs that are used by both the Database Engine and the SQL Server Native Client (SNAC)

- Supports 3 Client-Server Architectures types that employ one of the following protocols:
  - **Shared memory**
  - **TCP/IP**
  - **Named Pipes**

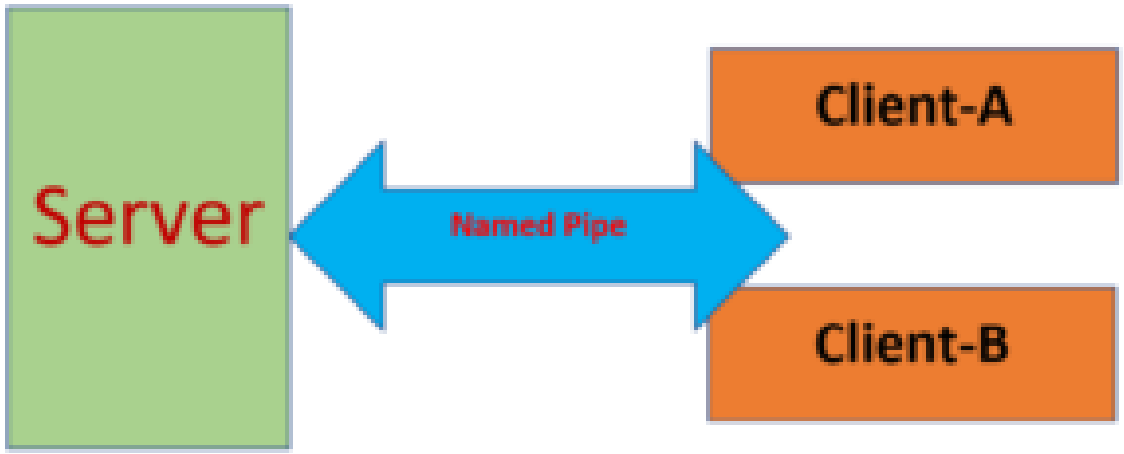
Microsoft SQL Server Architecture
Protocol Layer

- Shared memory
  - Is the default protocol used to connect from a client running on the same computer as Microsoft SQL Server.
  - Features:
    - Can only be used locally
    - Has no configurable properties
    - Is always tried first when connecting from the local machine



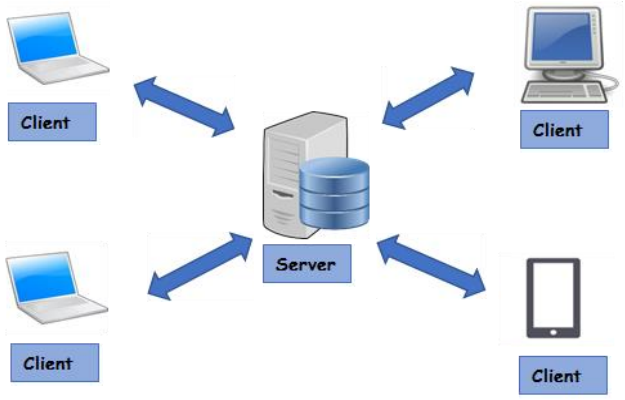
Microsoft SQL Server Architecture
Protocol Layer

- Named Pipes
  - TCP/IP and Named Pipes are comparable protocols in the architectures in which they can be used
  - Are used for local area networks (LANs)

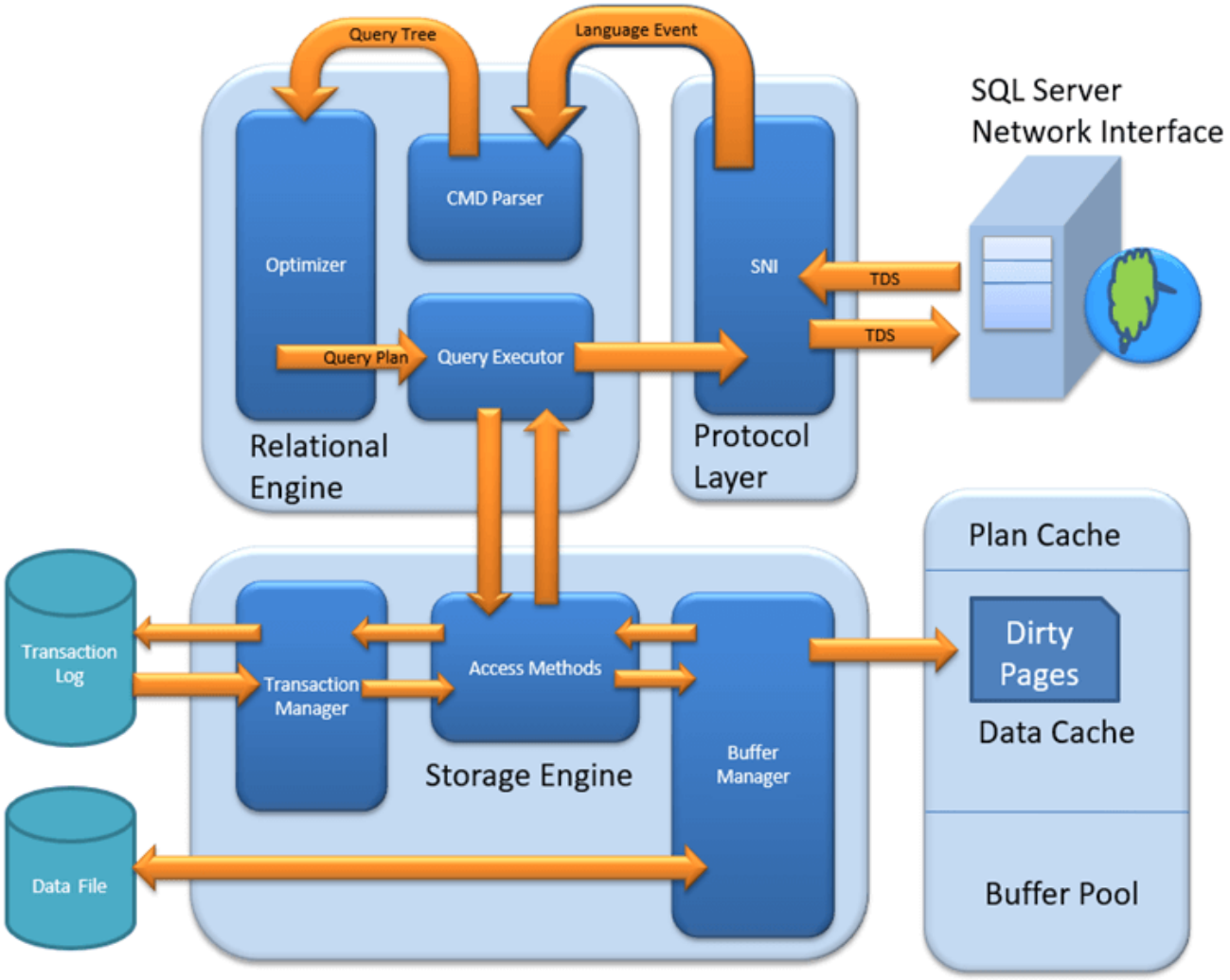


Microsoft SQL Server Architecture
Protocol Layer

- TCP/IP
  - The most used access protocol for MS SQL Server
  - Enables connection to MS SQL Server by specifying an IP address and a port number
  - The connection happens automatically when the instance to connect to is specify
  - The internal name resolution system resolves the hostname part of the instance name and gets the associated IP address
  - The connect either uses
    - The default TCP port number 1433 for default instances or
    - The SQL Browser service will find the right port for a named instance using UDP port 1434.

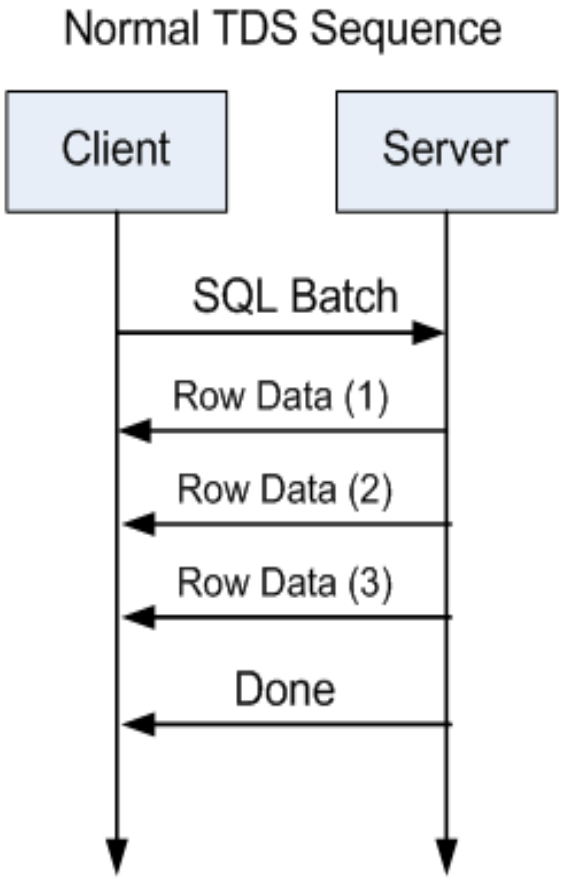


Microsoft SQL Server Architecture

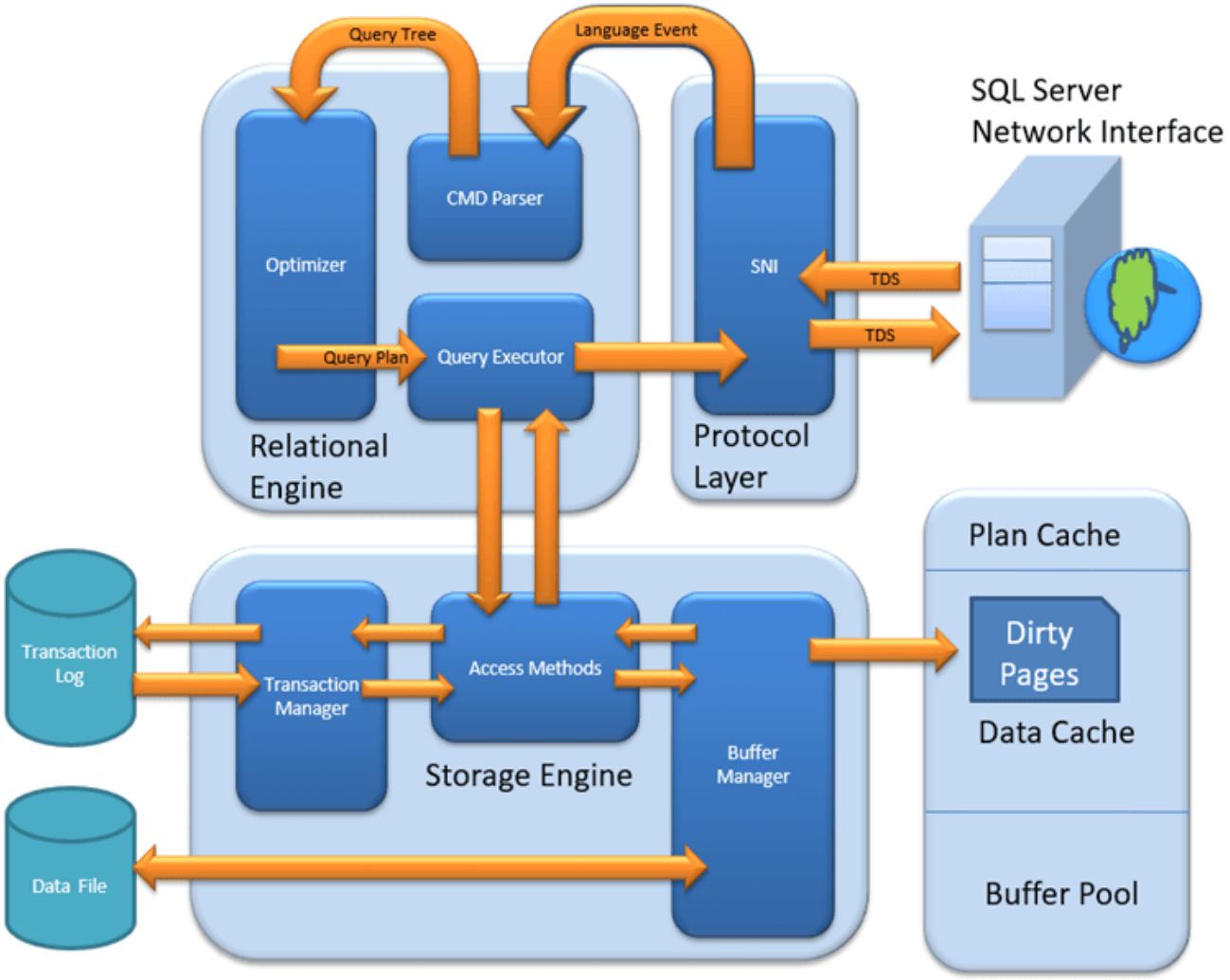


Microsoft SQL Server Architecture
Protocol Layer

- Tabular Data Stream (TDS) is a Microsoft proprietary protocol
- TDS were originally designed by Sybase
- TDS is used to interact with a database server
- All 3 protocols use TDS packets
- A TDS is encapsulated in Network packets
  - This enables data transfer from the client machine to the server machine
- Once a connection has been made using a network protocol (e.g., TCP/IP)
  - A link is established to the relevant TDS endpoint
  - TDS endpoint acts as the communication point between the client and the server



Microsoft SQL Server Architecture





Microsoft SQL Server Architecture
Relational Engine

- The Relational Engine
  - Is also sometimes called the query processor
  - Its primary function is query optimization and execution
  - It is responsible for the execution of user queries by
    - Requesting data from the storage engine
    - Processing the results that are returned

Microsoft SQL Server Architecture
Relational Engine

The main responsibilities of the relational engine are:

1. Parsing the SQL statements

- The parser scans an SQL statement
- Breaks down the SQL statement into
  - Logical units (i.e., keywords, parameters, operators, and identifiers)
  - A series of smaller logical operations

2. Optimizing the execution plans

- There are many ways that the server could use data from the source tables to build the result set
- The query optimizer determines
  - What these various series of steps are
  - Estimates the cost of each series (primarily in terms of file I/O)
  - Chooses the series of steps that has the lowest cost
  - Combines the specific steps with the **query tree** to produce an optimized execution plan



Microsoft SQL Server Architecture
Relational Engine

The main responsibilities of the relational engine are:

3. Executing the series of logical operations defined in the execution plan

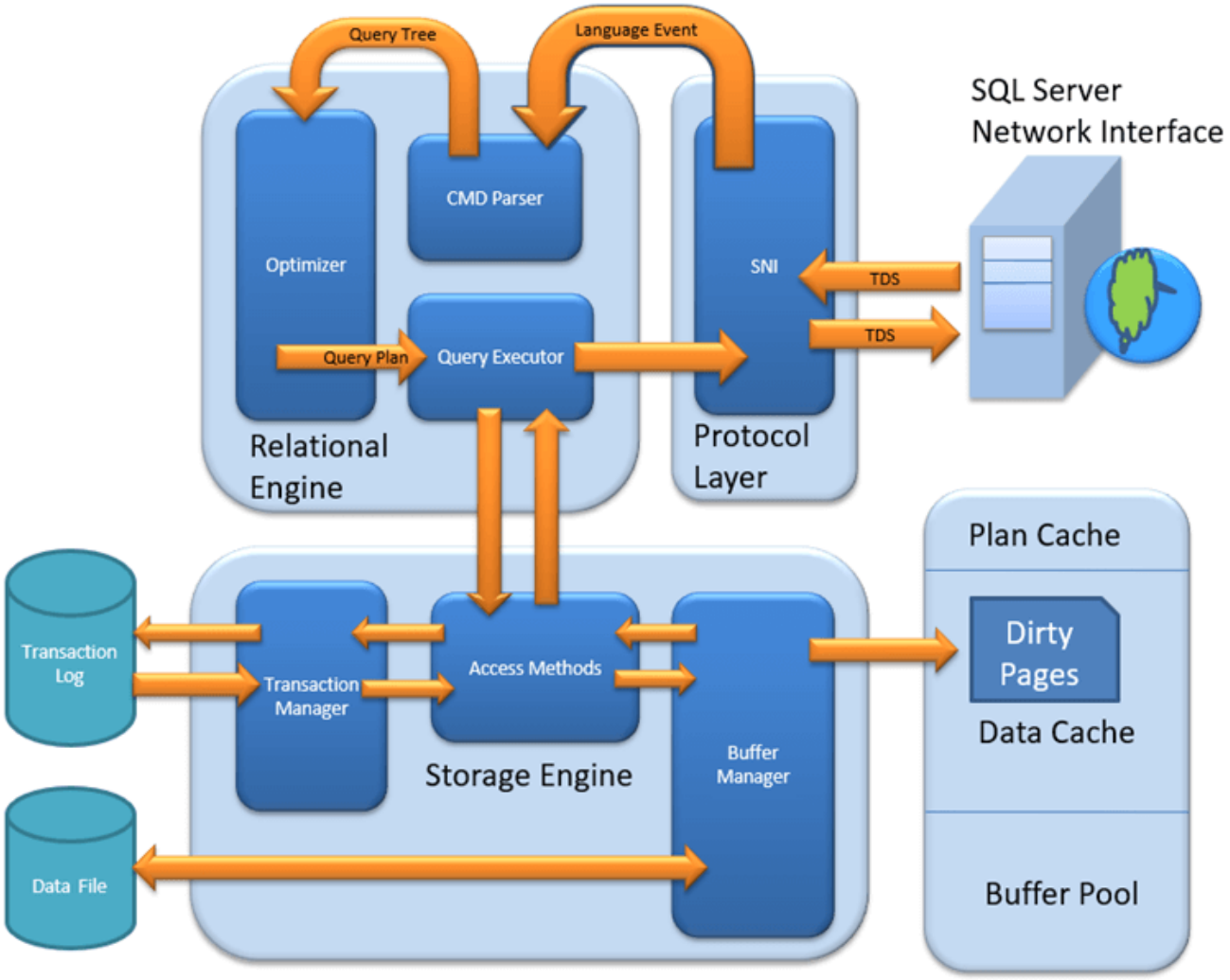
- After the query optimizer has defined the logical operations required to complete a statement, the relational engine steps through these operations in the sequence specified in the optimized execution plan

4. Processing Data Definition Language (DDL) and other statements.

5. Formatting results.

- The relational engine formats the results returned to the client
- The results are formatted as either
  - a traditional tabular result set
  - an XML document
- The results are then encapsulated in one or more TDS packets and returned to the application

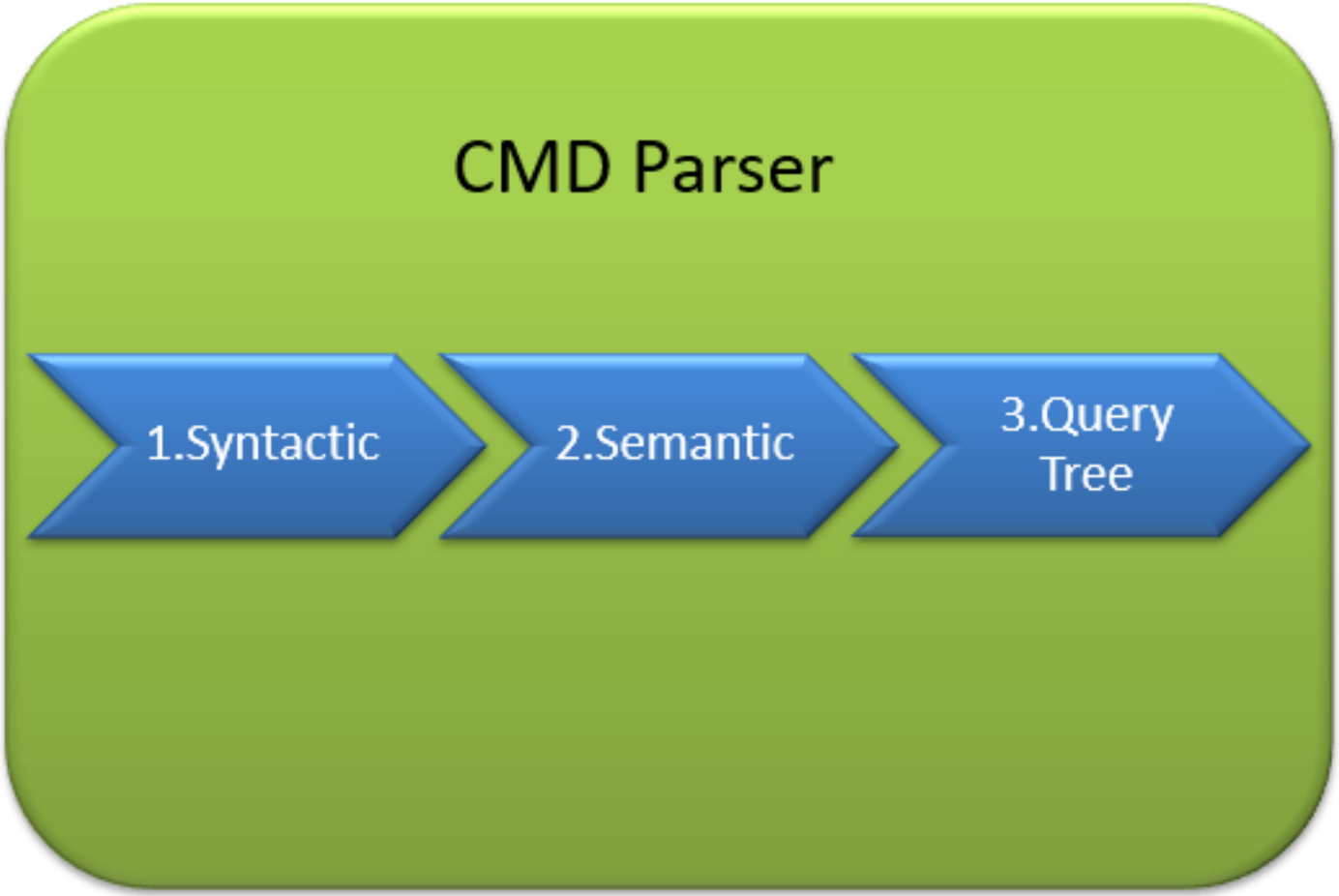
Microsoft SQL Server Architecture



Microsoft SQL Server Architecture
Relational Engine

- The Relational Engine contains below components:
  - **CMD Parser**
  - **Optimizer**
  - **Query Executor**

Microsoft SQL Server Architecture
Relational Engine

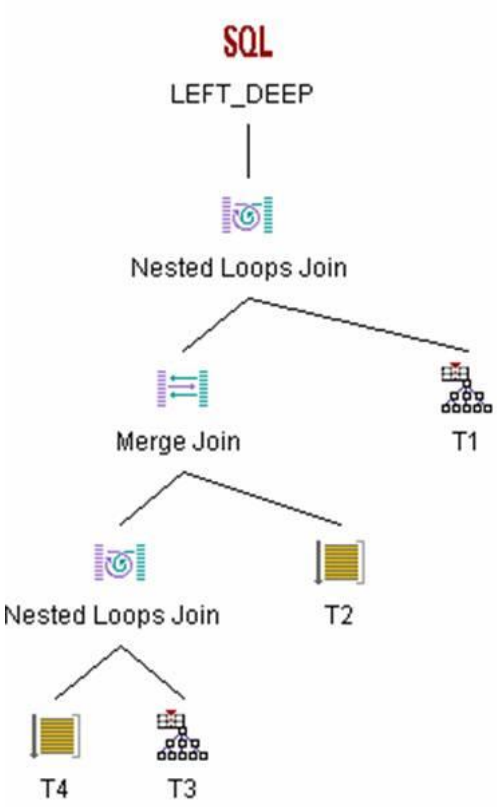


Microsoft SQL Server Architecture
Relational Engine

- **CMD Parser** (The Command Parser)
  - Handles SQL and Transact-SQL language
  - Steps:
    1. Checks the **syntactics and semantics of a query** and returns any errors back to the protocol layer to send to the client
    2. If the **syntax and semantics are valid**, then the next step is to generate a **query plan (query tree) or find an existing plan**.
    3. A **query plan** contains the details about how SQL Server is going to execute a piece of code (**execution plan**)
       - To check for a query plan, the Command Parser generates a **hash** of the SQL or Transact-SQL query
       - Checks the hash against the **plan cache** to determine whether a suitable plan already exists
    4. The **plan cache** is an area in the buffer pool used to cache query plans
       - If it finds a match, then the plan is read from cache and passed on to the Query Executor for execution
       - Otherwise, an execution plan is created by the optimizer

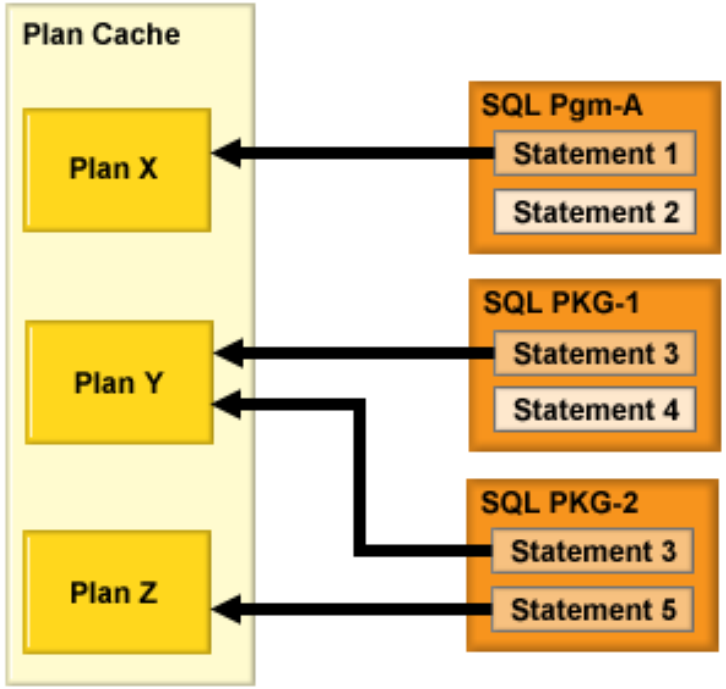
Microsoft SQL Server Architecture
Relational Engine

- **Execution Plan**
  - Is composed of primitive operations, some examples:
    - Reading a table completely
    - Using an index
    - Performing a nested loop or a hash join
  - All primitive operations
    - Have an output, i.e., their result set.
    - Some have one input, e.g., the nested loop
    - Other has two inputs, e.g., the hash join
  - Each input should be connected to the output of another primitive operation
  - **The execution plan can be sketched as a tree, i.e., information flows from leaves to the root.**



Microsoft SQL Server Architecture
Relational Engine

- **Plan Cache**
  - Is the part of Microsoft SQL Server's buffer pool
  - Is used to store execution plans in case they are needed later when the same type of scripts are submitted by the users.

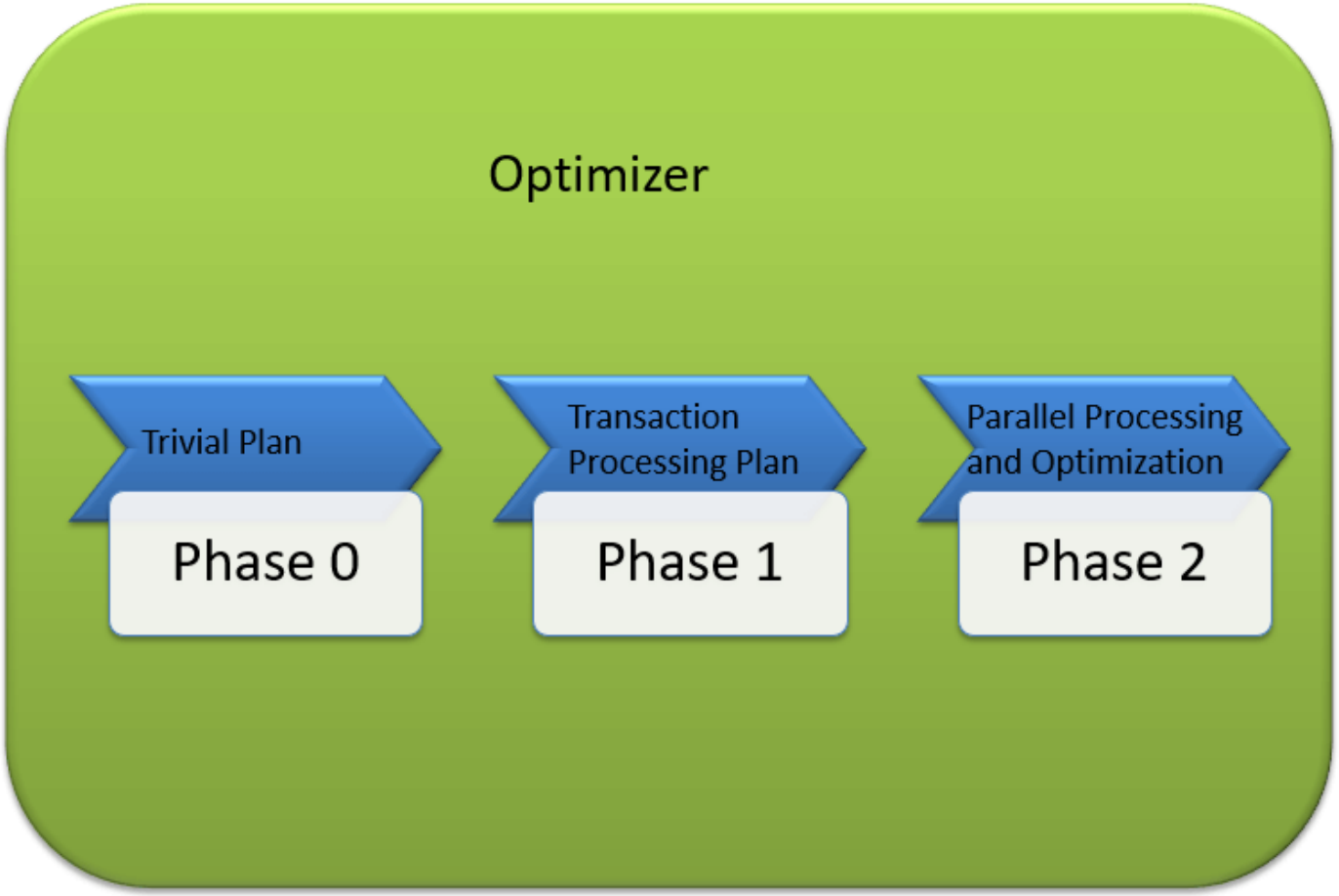




Microsoft SQL Server Architecture
Relational Engine

- **Optimizer**
  - Is one of the important assets of a database engine
  - Is the component on which a particular RDBMS stands off
  - Its primary function is to generate execution plan
  - Note that not all queries are optimized
  - Optimization is done for DML commands
    - DDL commands are not optimized, but they are instead compiled into an internal form
  - The query cost is calculated based on factors like CPU usage, Memory usage, and I/O needs
  - Its role is to find the **cheapest, not the best, cost-effective execution plan that minimizes the query run time**
  - **Works on inbuilt exhaustive/heuristic algorithms (Microsoft propriety)**

Microsoft SQL Server Architecture
Relational Engine





Microsoft SQL Server Architecture
Relational Engine

- **Optimizer Phase 0: Search for Trivial Plan**
  - This is also known as Pre-optimization stage
  - For some cases, there could be only one practical, workable plan, known as a trivial plan
  - In this case there is no need for creating an optimized plan
    - Searching for more plans would result
      - In finding the same run time execution plan
      - Lowers performance
  - If no Trivial plan found, then Phase 1 starts.



Microsoft SQL Server Architecture
Relational Engine

- **Optimizer Phase 1: Search for Transaction processing plans**
  - This includes the search for Simple and Complex Plans
  - Simple Plan
    - Analyses the past data of columns and indexes used in the query
    - Consists but is not restricted to analyze one index per table
  - Complex Plan involves analyzing multiple indexes per table, besides the use of past data

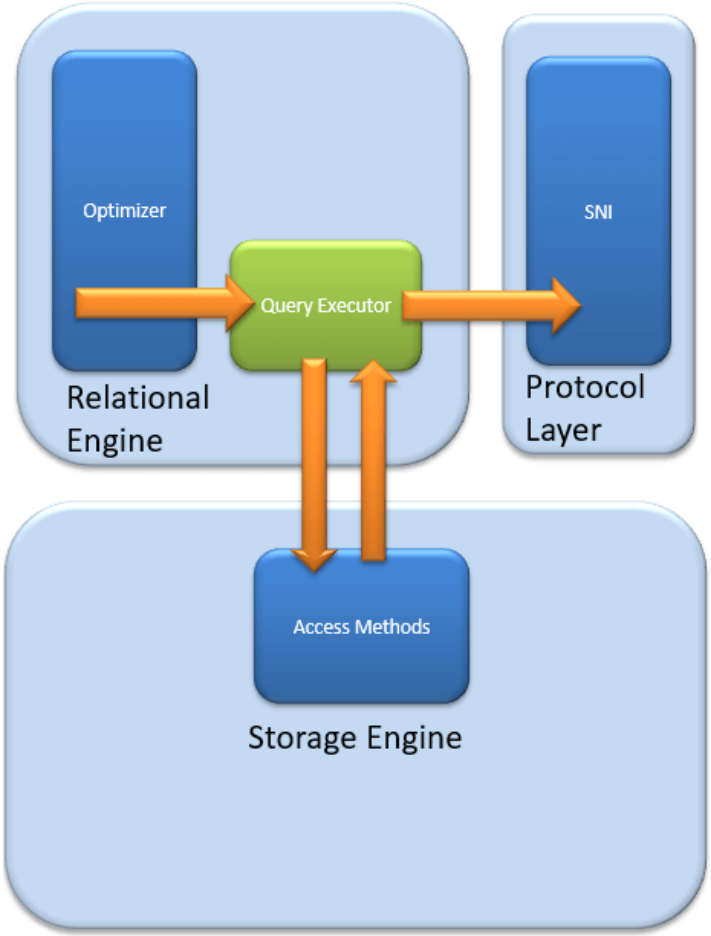


Microsoft SQL Server Architecture
Relational Engine

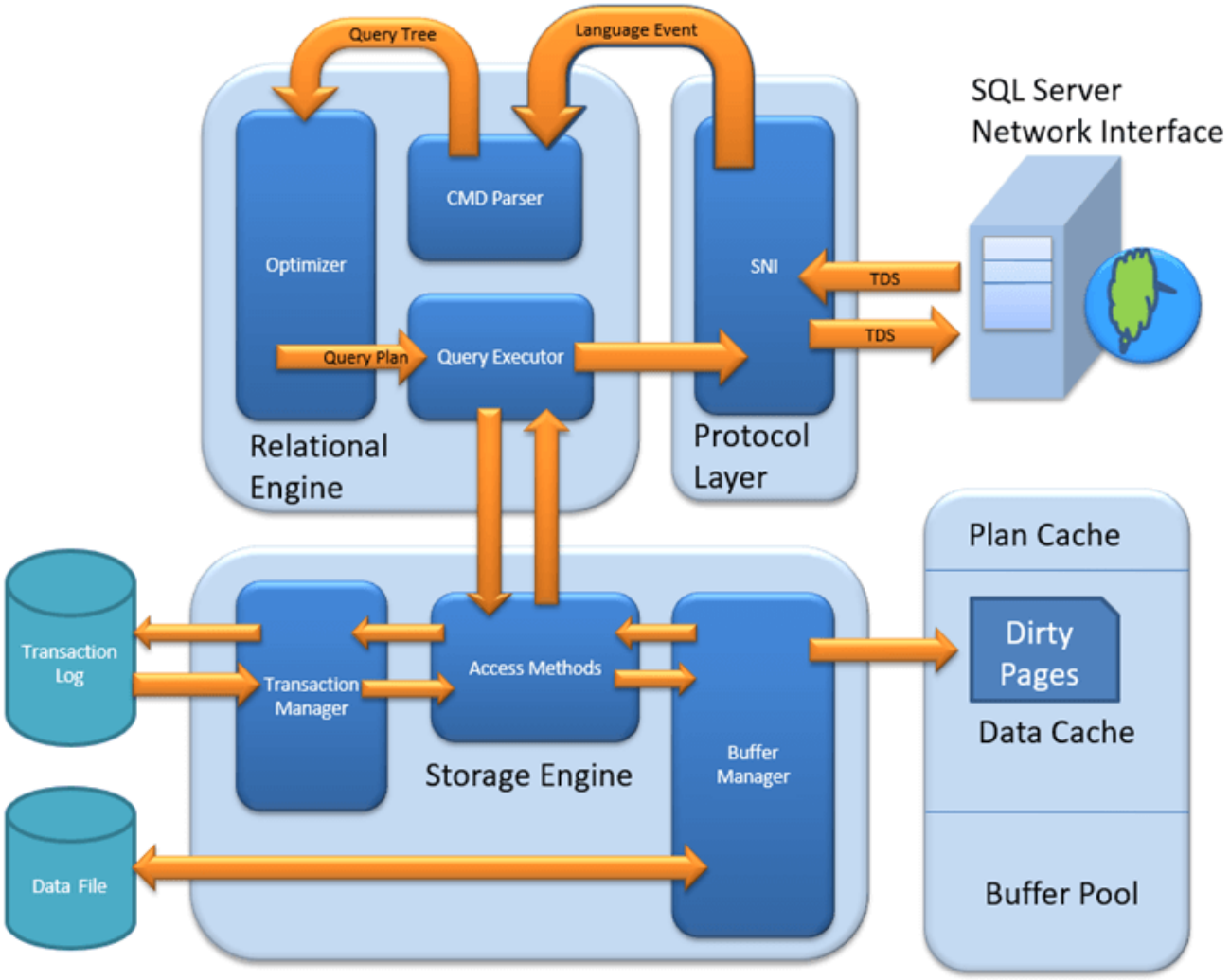
- **Optimizer Phase 2: Parallel Processing and Optimization**
  - If none of the above strategies work, the Optimizer searches for Parallel Processing possibilities
  - This depends on the Machine's processing capabilities and configuration
  - If that is still not possible, then the final optimization phase starts:
    - Finding all other possible options for executing the query in the best way

Microsoft SQL Server Architecture
Relational Engine

- **Query Executor**
  - Executes the query
    - More specific, it executes the query plan by working through each step it contains and interacting with the Storage Engine to retrieve or modify data
  - Calls the **Access Method**
  - Provides the execution plan for the logical requirement of fetching data
    - Once data is received from Storage Engine, the result gets published to the Protocol layer
    - Finally, data is sent to the end user using TDS



Microsoft SQL Server Architecture



Microsoft SQL Server Architecture
Storage Engine

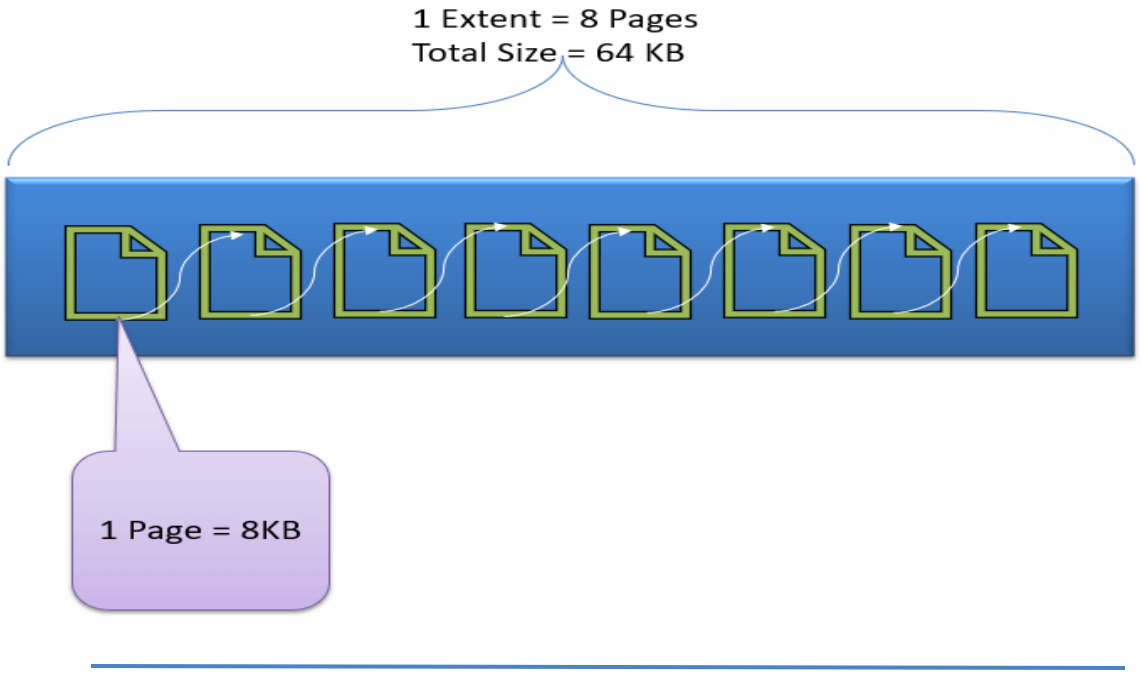
- The **Storage Engine** is responsible for managing all I/O to the data

- The main responsibilities of the storage engine include:
  - **Managing the files** on which the database is stored and managing the use of space in the files
  - **Building and reading the physical pages** used to store data
  - **Managing the data buffers** and **all I/O to the physical files**.
  - **Controlling concurrency**, i.e., managing transactions and using locking to control concurrent user access to rows in the database
  - **Logging and recovery**
  - Implementing utility **functions** such as the BACKUP, RESTORE, and DBCC statements, and bulk copy

Microsoft SQL Server Architecture
Storage Engine

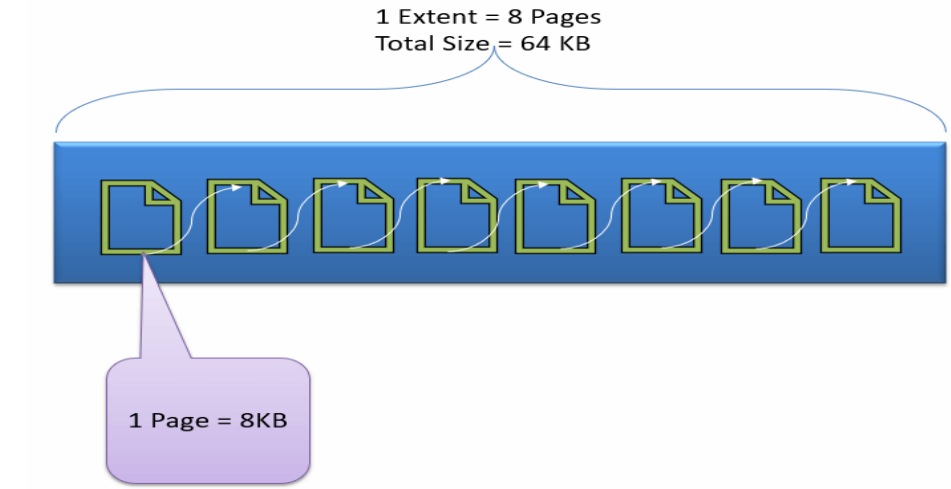
- The files in the storage engine are stored on Disk or SAN (Storage Area Network)

- The data is stored inside the database using Data Files



Microsoft SQL Server Architecture
Storage Engine

- Data File physically stores data in the form of data pages
- Each data page having a size of 8KB (forming the smallest storage unit in SQL Server)
- Consecutive data pages are logically grouped to form extents
- An extent is equal to 8 physically contiguous pages
- No object is assigned a page in SQL Server

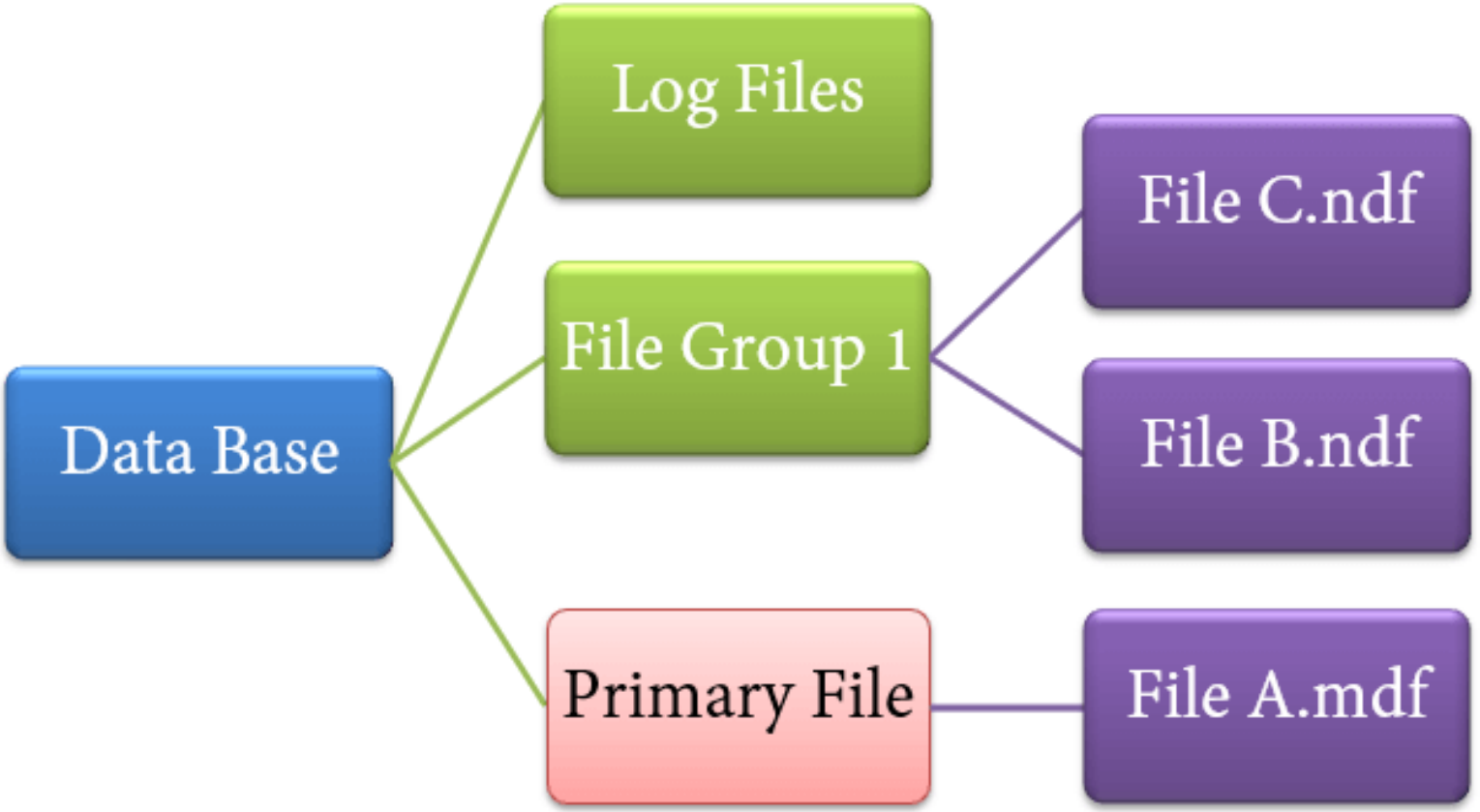




Microsoft SQL Server Architecture
Storage Engine

- The maintenance of the object is done via extents
- The page has a section called the Page Header with a size of 96 bytes
- The Page Header stores metadata information about the page:
  - Page Type
  - Page Number
  - Size of Used Space
  - Size of Free Space
  - Pointer to the next page and previous page

Microsoft SQL Server Architecture
Storage Engine





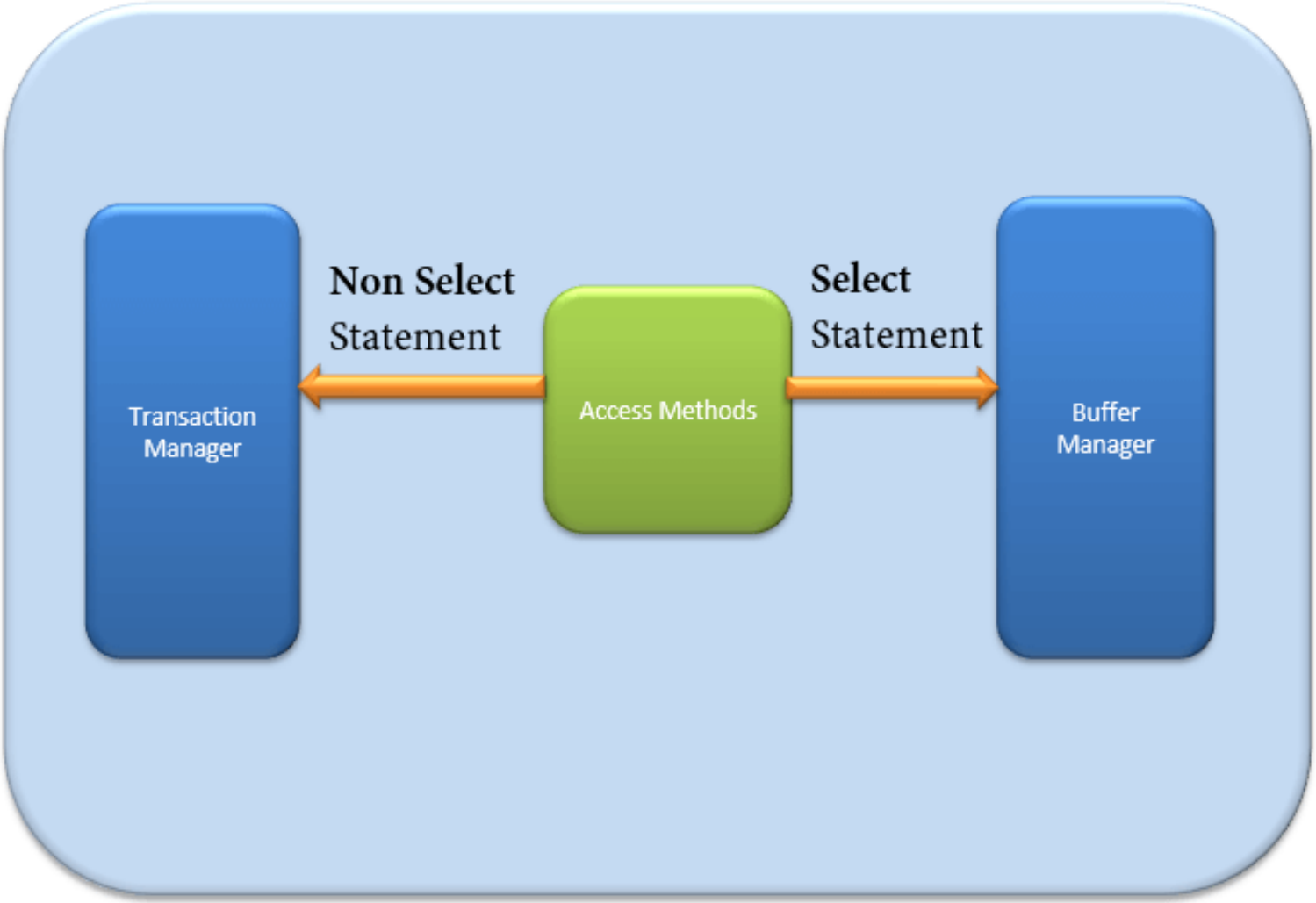
Microsoft SQL Server Architecture
Storage Engine

- Primary file
  - Every database contains one Primary file
  - This store all important data related to tables, views, Triggers, etc.
  - Extension is .mdf usually but can be of any extension
- Secondary file
  - A Database may or may not contains multiple Secondary files
  - This is optional and contains user-specific data
  - Extension is .ndf usually but can be of any extension
- Log file
  - Also known as Write Ahead Logs (WAL)
  - Extension is .ldf
  - Used for Transaction Management
    - To recover from any unwanted instances
    - To perform Rollback to uncommitted transactions

Microsoft SQL Server Architecture
Storage Engine

- The Storage Engine contains below components:
  - **Access Methods module**
  - **Buffer Manager**
  - **Transaction Manager**

Microsoft SQL Server Architecture
Storage Engine





Microsoft SQL Server Architecture
Storage Engine

- Access Methods act as an interface between query executor and Buffer Manager/Transaction Logs

- Access Methods provide
  - The storage structures for your data and indexes
  - The interface through which data is retrieved and modified

- Access Methods
  - Contain all the mechanisms and techniques to retrieve data
  - Does not perform the operation itself
  - The operation is passed as a request to
    - The Buffer Manager if the operation is a Select Statement
    - The Transaction Manager if the operation is not a Select Statement



Microsoft SQL Server Architecture
Storage Engine

- Buffer Manager consists of two mechanisms:
  - The Buffer Manager to access and update database pages
  - The Buffer Cache (also called the Buffer Pool), to reduce database file I/O

- The Buffer Manager manages the Buffer Pool (the majority of SQL Server's memory usage)
  - The Buffer Pool consist of various type of cache (e.g., data cache, plan cache, log cache, etc.)

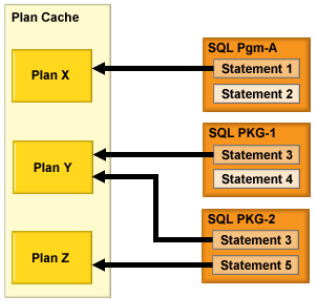
- Buffer Manager manages the core functions for:
  - Plan Cache
  - Data Cache
  - Dirty Page

Microsoft SQL Server Architecture
Storage Engine

- **Plan Cache** is used to store the query plan for a SQL query

- **Existing Query plan:**
  - The buffer manager checks if the execution plan is stored Plan Cache
  - If **Yes**, then query plan cache and its associated data cache is used

- **First time Cache plan:**
  - The buffer manager checks if the execution plan is stored Plan Cache
  - If **No,** then the first-time query execution plan is being run
  - After the run, the query plan store in the Plane Cache to ensure faster availability when the next time SQL server gets the same query
  - Therefore, it's nothing else but the query itself which plan execution is being stored if it is being run for the first time





Microsoft SQL Server Architecture
Storage Engine

- The **data cache** is the largest part of the Buffer Pool, thus, the largest memory consumer within SQL Server

- All the information read from disk is written in the data cache before use

- **Soft Parsing:**
  - When a query is issued, the Buffer Manager looks for Data in Buffer in Data Cache
  - If present, then this Data is used by the Query Executor
  - This improves the performance by reducing the number of I/O operation, i.e., faster to retrieve data from a cache as compared to fetching data from physical storage

- **Hard Parsing:**
  - When a query is issued and the data is not present in cache, then required Data is searched in Data Storage
  - After it is retrieved from the physical storage, the data is stored in the data cache for future use



Microsoft SQL Server Architecture
Storage Engine

- **Dirty pages**
  - When a page is read from disk into memory it is regarded as a clean page because it's the same as its counterpart on the disk
  - However, once the page has been modified in memory it is marked as a dirty page
  - A dirty page is simply a page that has changed in memory since it was loaded from disk and is now different from the on-disk page

Microsoft SQL Server Architecture
Storage Engine

- **Transaction Manager** interacts with the Access Methods and has two components
  - Lock Manager:
    - Responsible for providing concurrency to the data
    - Delivers the configured level of isolation by using locks
    - This process ensures **data consistency and isolation**
  - Log Manager writes the changes to the transaction log:
    - Keeps a track of all updates done in the system via logs in Transaction Logs
    - Logs have **Logs Sequence Number with the Transaction ID and Data Modification Record**.
    - This is used for keeping track of **Transaction Committed and Transaction Rollback**



Microsoft SQL Server Architecture
Storage Engine

- Execution Process
  - Log Manager start logging and Lock Manager locks the associated data.
  - A copy of the data is maintained in the Buffer cache
  - A copy of data supposed to be updated is maintained in Log buffer
  - Modification are applied on the data stored in Data buffer resulting in Dirty Pages.
  - Checkpoint and Write-Ahead Logging:
    - This process writes all the modified page (Dirty Pages) from memory to Disk, while keeping them in the cache
    - A page is first pushed to the data page of the log file from Buffer log before becoming persistent on the disk
    - This is known as Write Ahead Logging
    - Frequency is approximately 1 run per minute
  - Lazy Writer:
    - The Dirty page can remain in memory
    - When SQL server observes a huge load and Buffer memory is needed for a new transaction, it frees up Dirty Pages from the cache
    - It operates on LRU (Least Recently Used) Algorithm for cleaning page from buffer pool to disk



Overview

- Microsoft SQL Server Overview
- Microsoft SQL Server Architecture
- Microsoft SQL Server Big Data Cluster

SQL Server Big Data Cluster

- SQL Server Big Data Cluster enabled with the release of Microsoft SQL Server 2019

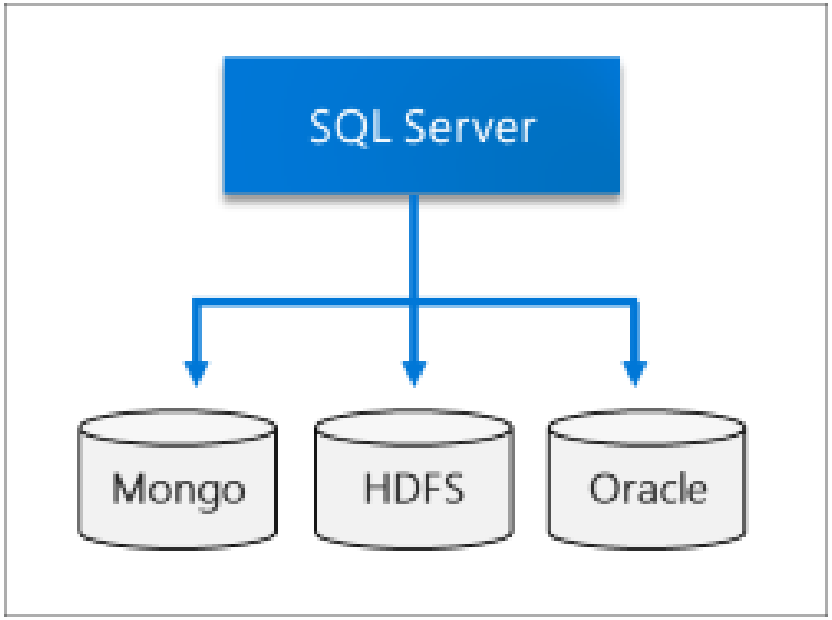
- SQL Server Big Data Cluster offers:
  - Data Virtualization
  - Data Lake
  - Scale out Data Mart
  - Integrated Artificial Intelligence and Machine Learning Tools
  - Management and Monitoring

- A SQL Server big data cluster is a cluster of Linux containers orchestrated by Kubernetes.

SQL Server Big Data Cluster
Data Virtualization

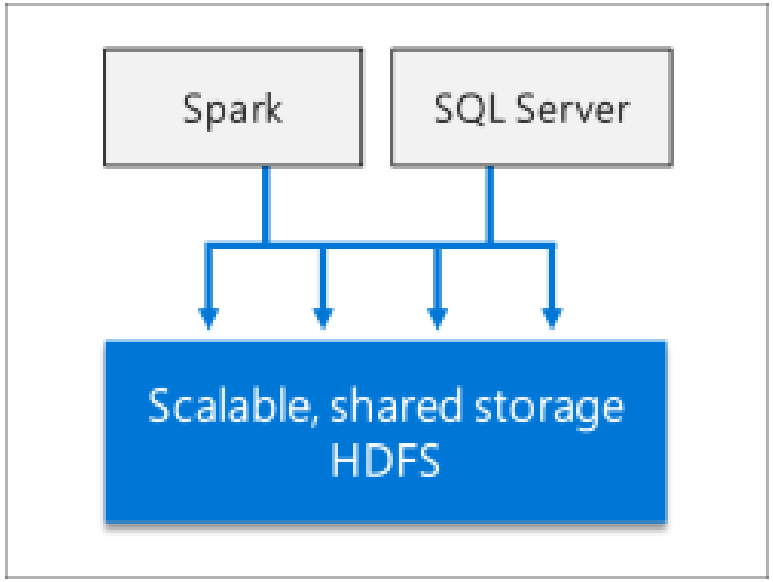
- SQL Server Big Data Clusters can query external data sources without moving or copying the data

- SQL Server 2019 (15.x) introduces new connectors to data sources



SQL Server Big Data Cluster
Data Lake

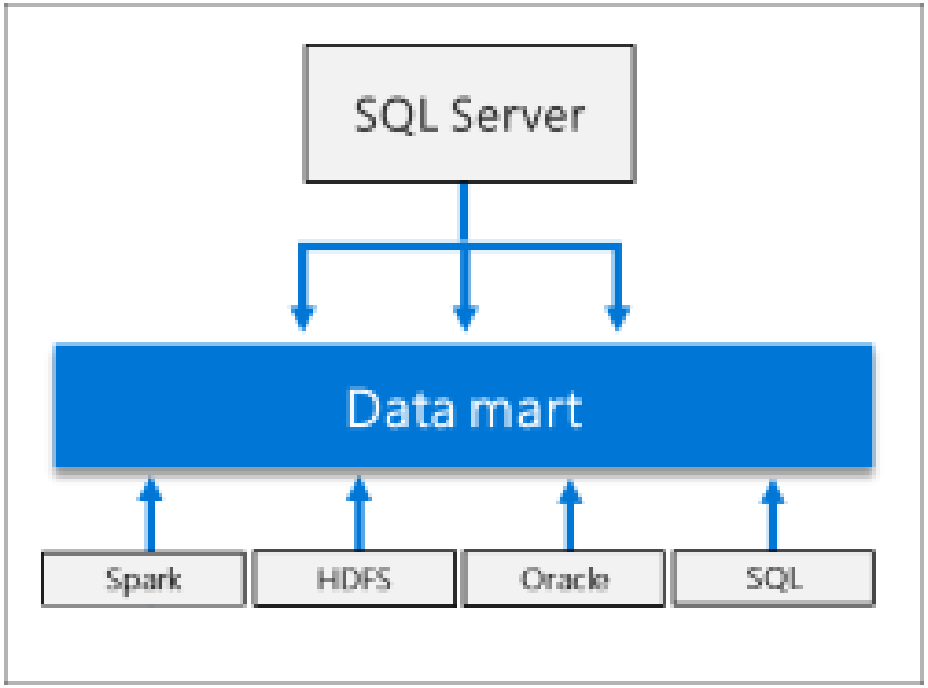
- SQL Server Big Data Cluster includes a scalable HDFS *storage pool*
- The HDFS *storage pool* can be used to store big data (potentially ingested from multiple external sources)
- Once the data is stored in HDFS in the big data cluster, it can be
  - Analyzed
  - Queried
  - Combined with relational data.



SQL Server Big Data Cluster
Scale-out Data Mart

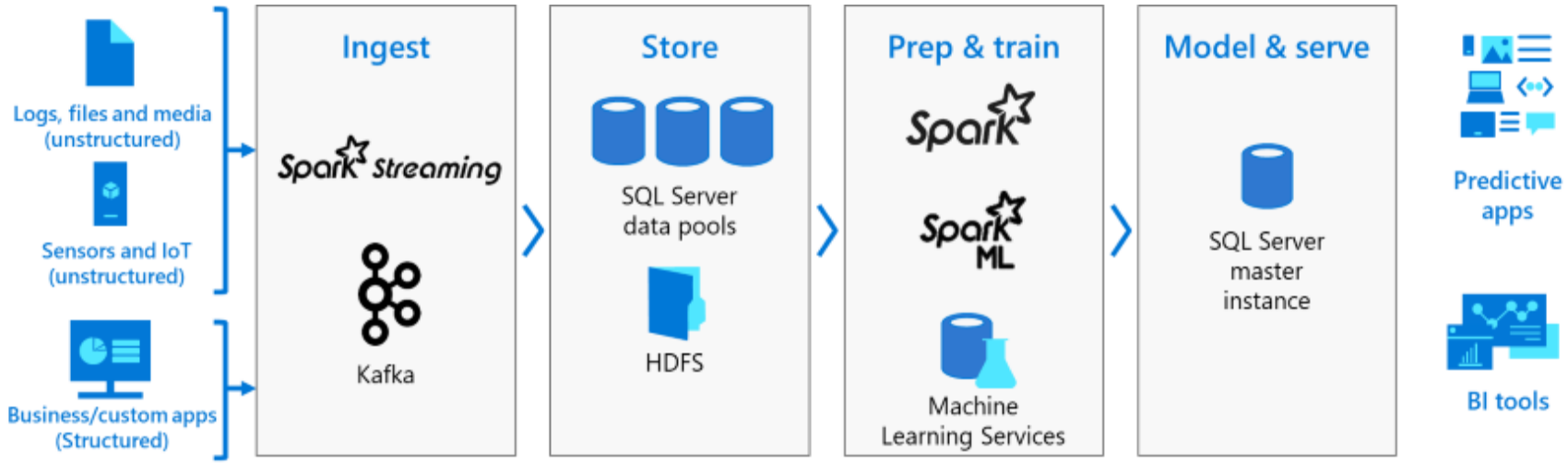
- SQL Server Big Data Clusters provide scale-out compute and storage to improve the performance of analyzing any data

- Data from a variety of sources can be ingested and distributed across *data pool* nodes as a cache for further analysis



Big Data Cluster
Integrated AI and ML Tools

- SQL Server Big Data Clusters enable AI and ML tasks on the data stored in HDFS storage pools and the data pools:
  - Spark
  - Built-in AI tools in SQL Server, using R, Python, Scala, or Java.



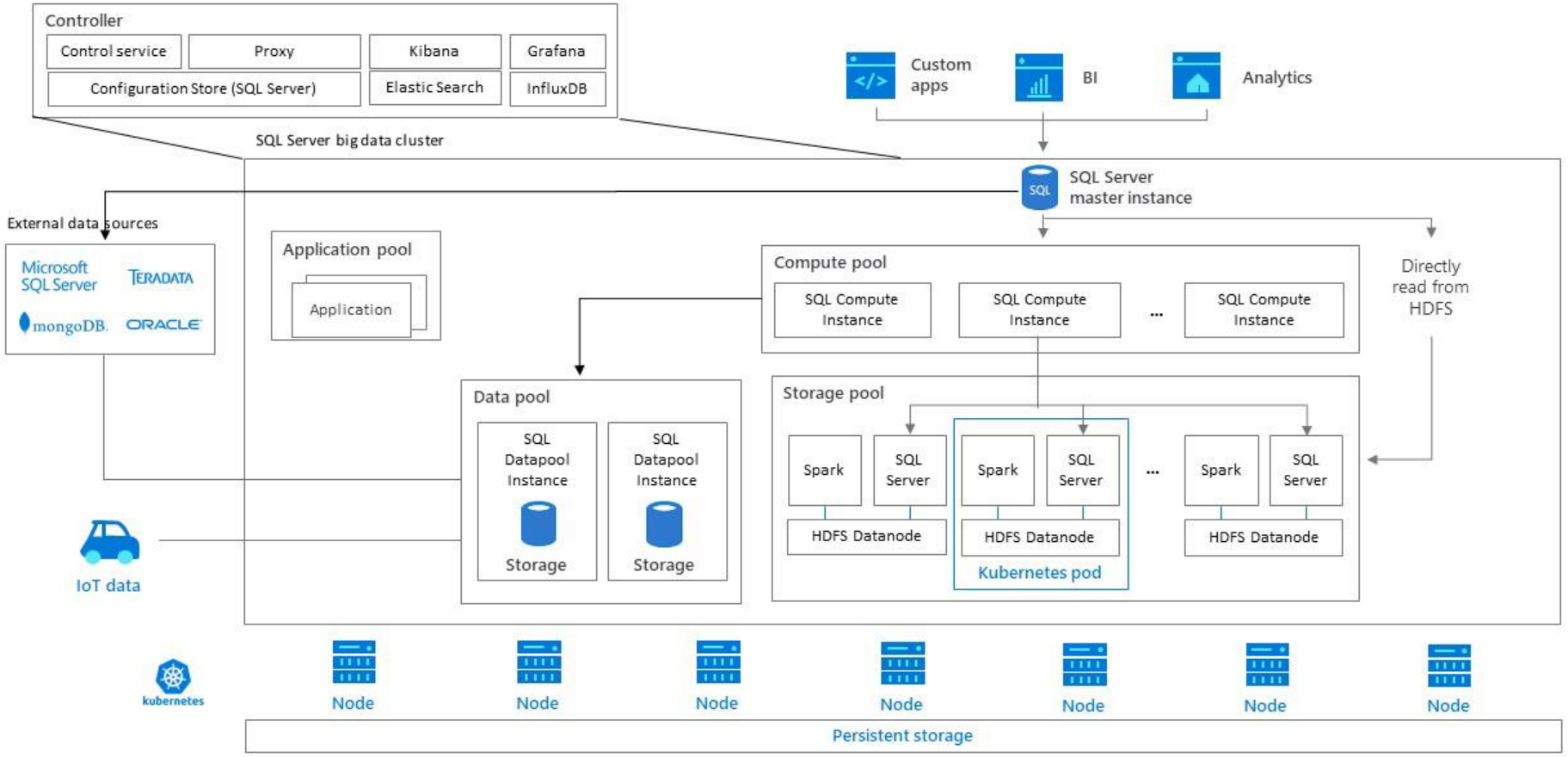


SQL Server Big Data Cluster Management and Monitoring

- Management and monitoring are provided through:
  - Command line tools
  - APIs
  - Portals
  - Dynamic management views

- Azure Data Studio can be used to perform a variety of tasks on the cluster:
  - Built-in tools for common management tasks.
  - Ability to browse HDFS, upload files, preview files, and create directories
  - Ability to create, open, and run Jupyter-compatible notebooks
  - Data virtualization wizard to simplify the creation of external data sources (enabled by the Data Virtualization Extension)

SQL Server Big Data Cluster



SQL Server Big Data Cluster

- Controller
  - Provides management and security for the cluster
  - Contains
    - The control service
    - The configuration store
    - Other cluster-level services such as Kibana, Grafana, and Elastic Search.

- Compute pool
  - Provides computational resources to the cluster
  - Contains nodes running SQL Server on Linux pods
  - The pods in the compute pool are divided into SQL Compute instances for specific processing tasks

SQL Server Big Data Cluster

- Data pool
  - Is used for data persistence and caching
  - Consists of one or more pods running SQL Server on Linux
  - Is used to ingest data from SQL queries or Spark jobs
  - SQL Server big data cluster data marts are persisted in the data pool
- Storage pool
  - Consists of storage pool pods comprised of SQL Server on Linux, Spark, and HDFS
  - All the storage nodes in a SQL Server big data cluster are members of an HDFS cluster
  - Kubernetes can be used to create Storage pods

SQL Server Big Data Cluster

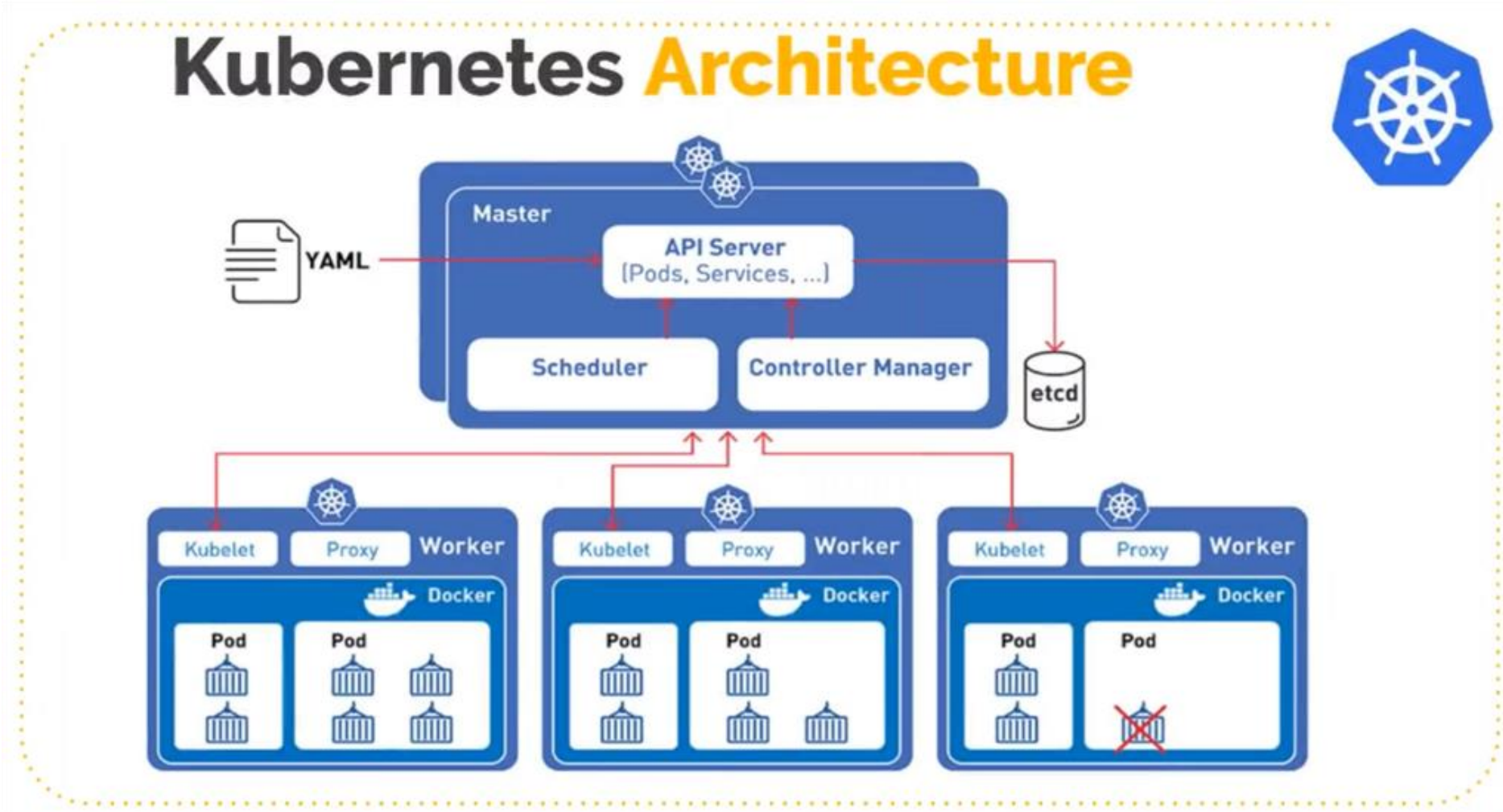
- Kubernetes is an open-source container orchestrator
- Kubernetes scales container deployments according to needs
- Kubernetes Cluster is a set of machines (nodes)
  - 1 node is the Master node
    - Controls the cluster
    - Distributes the work between the workers
    - Monitors the health of the cluster
  - The remaining nodes are the workers

SQL Server Big Data Cluster

- Kubernetes Node
  - Runs containerized applications
  - Can be either a physical machine (physical node) or a virtual machine (virtual node)
  - A Kubernetes cluster contains a mixture between physical and virtual nodes

- Kubernetes Pod
  - Is the atomic deployment unit of Kubernetes
  - Is a logical group of one or more containers and associated resources needed to run an application
  - Each pod runs on a node
  - A node can run one or more pods
  - The Kubernetes master automatically assigns pods to nodes in the cluster

SQL Server Big Data Cluster



References

- Microsoft SQL Server 2019 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver15

- Microsoft SQL Server 2022 Technical Documentation
https://learn.microsoft.com/en-us/sql/sql-server/?view=sql-server-ver16